MoonWalk

Nanopayment rails on Arc. One agent wallet, many people, each person's spend limit enforced by a contract.

Live on

Arc testnet, chain 5042002. Three contracts, deployed and in use.

Surface

A public Discord agent that decides what to buy and pays for it in USDC.

Proof

Every number here has a transaction hash in the repo.

A $0.001 payment costs $0.0019 to settle

Metered agent payments break at the prices that make metering worth doing. On Arc, one x402 settlement of a $0.001 call used 87,145 gas, which is $0.001873. The fee is bigger than the payment.

So builders take one of two exits. Pay the fee anyway. Or keep the meter in their own database and settle later. The second means every user trusts the operator's arithmetic.

And it gets worse in a group

A shared assistant has one wallet and many people behind it. The chain sees one payer. So a per-person limit can only live in the operator's backend, where it is a promise, not a rule.

What the field does

38 projects in this hackathon claim spend caps. Five claim them on-chain. Every one of them caps a single owner's budget.

Sign a cumulative total, settle the batch

The payer funds a channel once with an EIP-3009 `receiveWithAuthorization`: no approve, no gas, no transaction, just a signature. Every call after that costs one EIP-712 voucher carrying a running total for one subject. The service keeps the newest voucher and redeems a batch in a single transfer.

Per call, alone

$0.001873

87,145 gas, one transaction per call

Per call, batched

$0.000219

30 calls in one transaction, 262,639 gas

Break-even

4 calls

Above that the channel is cheaper. The gap grows with volume

A replayed voucher pays nothing, because the contract stores the highest total settled and pays only the difference. A lost voucher costs nothing either.

Put the limit in consensus, not in the app

A subject is `keccak256("discord:<guildId>:<userId>")`. Each person in a shared room gets their own cap in the SpendGuard contract while the agent keeps spending from one wallet.

The channel asks the guard on every redeem. A voucher over someone's cap cannot be redeemed by anyone, including us. In the live run, a voucher $0.001 over a $0.005 cap came back as `CapExceeded`, from the chain, not from our backend.

Who can change a cap

The cap owner, which can be delegated away from the payer. A Discord admin sets a member's cap with `/cap` and the change lands in the contract.

What a cap cannot do

Authorize spending. Caps only restrict. Every payment still needs the payer's own signature, so delegating cap administration is safe.

EVIDENCE

Every claim has a transaction

WHAT	RESULT	TRANSACTION
Channel opened by a signature alone	$0.20 deposit, payer sent nothing	`0x9923efff…`
30 metered calls redeemed	$0.03 in one transfer, 262,639 gas	`0xb779492a…`
Voucher over a cap	CapExceeded, refused before any transaction	no transaction to show
Closed with both signatures	$0.17 returned, submitted by the service	`0x622b3619…`
Through the running API, two users	6 calls, one settlement, per-person totals matched	`0xfda3b8a9…`
A member lists a priced service from Discord	$0.0010, member's wallet as payTo, not yet buyable	`0x51f12944…`
An admin approves it	Buyable from that block, and a reprice drops it	`0xf07f9c2a…`

Payer transactions sent

0

Before and after every run

Tests

51 + 199

Foundry and pytest, mypy strict clean

Contracts live

3

Channel, guard, registry

Used, not named

MoonWalk already ran on Circle rails: USDC on Arc, EIP-3009 authorizations, x402 over HTTP. Two of Circle's own services are now in the code and exercised live, and four more were tried and rejected with a written reason.

Developer-controlled Wallets

The signer is an interface, so the key can live inside Circle instead of this process. Circle signed a channel voucher for chain 5042002 and the digest matched the channel's own `voucherHash()` view on Arc byte for byte, then signed an EIP-3009 authorization a relayer submitted. USDC left the custodied wallet with that wallet sending no transaction and paying no gas.

CCTP V2 with Iris

The agent refills its own Arc balance: burn on Ethereum Sepolia, wait for the attestation, mint on Arc. Three real transfers, six transactions, one of them decided by the script itself when the balance fell under its threshold. Circle charged $0.00005 of the $0.000063 allowed.

WHAT	RESULT	TRANSACTION
Circle's signature, submitted by a relayer	USDC moved, wallet transaction count 0	`0xfd1742ab…`
Threshold-driven refill, Sepolia to Arc	$0.49995 minted, mint gas paid in USDC	`0xe65307d2…`

Gateway is read only here on purpose, so the comparison with our own channel is written against real numbers. Gateway Nanopayments, the Faucet API, Paymaster and StableFX each have a reason for not being used, in docs/CIRCLE-INTEGRATIONS.md.

The only entry where one wallet serves a room

Scanned all 1073 projects in this hackathon. Plenty batch payments. Plenty cap an agent. All 13 chat-surface projects chose Telegram or WhatsApp, where every user has their own wallet, so the problem never appears.

MoonWalk is the one where a single agent wallet serves many distinct humans in a shared room and each person's limit is enforced by a contract rather than by the backend.

Honest about the overlap

Circle Gateway Nanopayments nets a payer's outflows too. It meters a payer, not the many people behind one payer. It also needs a deposit into Circle's contract with a TEE computing batches. Our deposit sits in a contract whose source is in the repo. The 402 can offer both and let the agent choose.

Honest about the limits

Testnet only, since Arc mainnet is not live. The channel nets one payer to one service. The service holds the vouchers, so losing them costs the service what it has not collected yet. It recovers the rest from chain state.

What we would build with the accelerator

- Settle on a schedule as well as a threshold, so a service can choose its own counterparty risk.
- Multilateral netting, so services paying each other inside one settlement stop being a special case.
- A public meter page, so a member can audit their own spend without asking the operator.
- The same rails behind Slack and Telegram, since the subject is a hash of platform, room and person rather than an address.

NanoChannel

`0x3e2dE84eD534E39241682957d617ed761892D568`

SpendGuard

`0xfbB8e1E61e8FbB09e5d5be308ac4F54D2865B67b`

ServiceRegistry

`0x774E5F27b572450F5D21FE3929B45557F3468F9b`